# Log, Trace, and Monitor - Docs by LangChain

4–5 minutes

---

When building apps or agents using LangChain, you end up making multiple API calls to fulfill a single user request. However, these requests are not chained when you want to analyse them. With **Portkey**, all the embeddings, completions, and other requests from a single user request will get logged and traced to a common ID, enabling you to gain full visibility of user interactions. This notebook serves as a step-by-step guide on how to log, trace, and monitor LangChain LLM calls using `Portkey` in your LangChain app. First, let's import Portkey, OpenAI, and Agent tools

```
import os

from langchain.agents import AgentExecutor, create_openai_tools_agent
from langchain_openai import ChatOpenAI
from portkey_ai import PORTKEY_GATEWAY_URL, createHeaders
```

Paste your OpenAI API key below. (You can find it here)

```
os.environ["OPENAI_API_KEY"] = "..."
```

## Get Portkey API Key

1. Sign up for Portkey here
2. On your dashboard, click on the profile icon on the bottom left, then click on “Copy API Key”
3. Paste it below

```
PORTKEY_API_KEY = "..."  # Paste your Portkey API Key here
```

## Set Trace ID

1. Set the trace id for your request below
2. The Trace ID can be common for all API calls originating from a single request

```
TRACE_ID = "uuid-trace-id"  # Set trace id here
```

```
portkey_headers = createHeaders(
    api_key=PORTKEY_API_KEY, provider="openai", trace_id=TRACE_ID
)
```

Define the prompts and the tools to use

```
from langchain_classic import hub
from langchain.tools import tool

prompt = hub.pull("hwchase17/openai-tools-agent")



@tool
def multiply(first_int: int, second_int: int) -> int:
```

```
    """Multiply two integers together."""
    return first_int * second_int


@tool
def exponentiate(base: int, exponent: int) -> int:
    "Exponentiate the base to the exponent power."
    return base**exponent


tools = [multiply, exponentiate]
```

Run your agent as usual. The **only** change is that we will **include the above headers** in the request now.

```
model = ChatOpenAI(
    base_url=PORTKEY_GATEWAY_URL, default_headers=portkey_headers,
temperature=0
)

# Construct the OpenAI Tools agent
agent = create_openai_tools_agent(model, tools, prompt)

# Create an agent executor by passing in the agent and tools
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
```

```
agent_executor.invoke(
    {
        "input": "Take 3 to the fifth power and multiply that by thirty
six, then square the result"
    }
)
```

```
> Entering new AgentExecutor chain...

Invoking: `exponentiate` with `{'base': 3, 'exponent': 5}`



243
Invoking: `multiply` with `{'first_int': 243, 'second_int': 36}`



8748
Invoking: `exponentiate` with `{'base': 8748, 'exponent': 2}`


76527504The result of taking 3 to the fifth power, multiplying it by 36,
and then squaring the result is 76,527,504.

> Finished chain.
```

```
{'input': 'Take 3 to the fifth power and multiply that by thirty six,
```

```
then square the result',
 'output': 'The result of taking 3 to the fifth power, multiplying it by
36, and then squaring the result is 76,527,504.'}
```

## How Logging & Tracing Works on Portkey

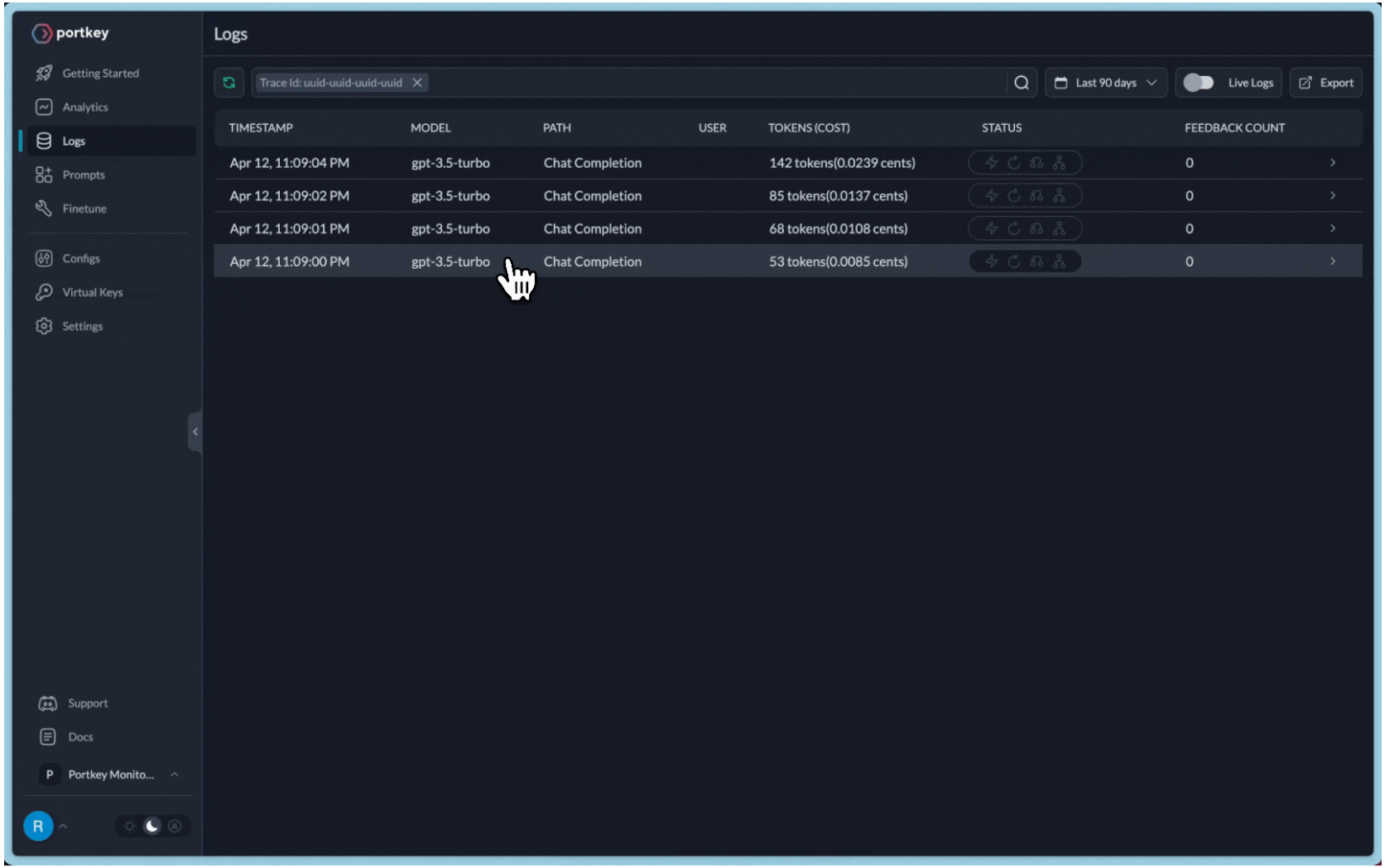
### Logging

- Sending your request through Portkey ensures that all of the requests are logged by default
- Each request log contains `timestamp`, `model name`, `total cost`, `request time`, `request json`, `response json`, and additional Portkey features

[Tracing]

- Trace id is passed along with each request and is visible on the logs on Portkey dashboard
- You can also set a **distinct trace id** for each request if you want
- You can append user feedback to a trace id as well. [More info on this here]

For the above request, you will be able to view the entire log trace like this




## Advanced LLMOps Features - Caching, Tagging, Retries

In addition to logging and tracing, Portkey provides more features that add production capabilities to your existing workflows: **Caching** Respond to previously served customers queries from cache instead of sending them again to OpenAI. Match exact strings OR semantically similar strings. Cache can save costs and reduce latencies by 20x. Docs **Retries** Automatically reprocess any unsuccessful

API requests **`upto 5`** times. Uses an **`exponential backoff`** strategy, which spaces out retry attempts to prevent network overload. Docs **Tagging** Track and audit each user interaction in high detail with predefined tags. Docs

---